

Vox Conversation Mode: A Z Specification for the Call State Machine

Formal Model of idle/listening/waiting/speaking

August 2026

Contents

1	Introduction	2
2	Basic Types	2
3	Free Types	2
4	Which Detector Governs Each Mode	3
5	Pending Addenda	3
6	Initialisation	3
7	Operations	3
7.1	Call lifecycle	3
7.2	Turn detection	4
7.3	The agent's turn	4
7.4	Observation	5
8	The Single Serialized Dispatch Point	5
9	Playback Sink Ordering	6
9.1	Basic Types and State	6
9.2	Initialisation	6
9.3	Operations	7
9.4	The First Open Question: Natural Completion vs. Barge-In	7
9.5	The Second and Third Open Questions: Actor, Not Lock	7
10	Known Limitations	8

1 Introduction

A Conversation Mode call moves through four modes – *idle* (no call), *listening* (waiting to hear the human), *waiting* (a turn has been sent to the agent; no reply audio yet), and *speaking* (the agent’s reply is playing). This document models that machine formally, per `docs/WORKFLOW.md`’s stateful-audio gate: the machine has four modes and transitions between them, invariants must hold across those transitions (a pending human utterance is never silently dropped), and a wrong transition here is exactly the class of defect that gate exists to catch before code is written, not after.

Two questions were named in `docs/conversation-mode-prd.tex` as requiring resolution *before* this model was written, not discovered while writing it (Chapter 2, “Known unknowns”; Chapter 3, §?? item 6). Both are resolved here, explicitly:

1. **Which detector governs a human speaking during the agent’s silent think-time (*waiting*, before any reply audio starts)?** Resolved in §4 as a totally-defined function from Mode to Detector: the *turn detector* governs both *listening* and *waiting* – barge-in detection requires agent audio to interrupt, and no agent audio exists until *speaking* begins, so there is nothing for a barge-in detector to do during *waiting*. A human who speaks again during *waiting* is not interrupting anything; the turn detector’s normal end-of-utterance logic applies, and what it captures is held as a *pending addendum* (§5), not discarded and not immediately dispatched as a second, concurrent turn (FR-4: one call is one human and one agent session at a time; the current turn is still in flight).
2. **Where does the single serialized dispatch point for this state machine live?** Resolved in §8 as a design requirement on the machine’s *caller*, not a property the pure state schema below can enforce by itself – exactly as `voxd/programs/program.py`’s `Program` already documents for its own single-writer assumption. §8 states the resolution and why it is the right one for this codebase.

This model does not attempt session attachment, turn-detection acoustics, or synthesis timing – those are described informally in `docs/conversation-mode-prd.tex` Chapter 2 and are the subject of Slice 1b onward. This model’s job is the state machine those components drive: the shape `Program` already proved out for Audio Programs (`docs/audio-programs.tex`), applied to a live call instead of a playlist.

2 Basic Types

There is no session or agent identifier in this model. FR-4 requires exactly one human and one Claude Code agent session per call, and that constraint is structural (one `CallState` instance per call, created by `StartCall` and destroyed by the return to *idle*) rather than a value carried in the state, the same reasoning `docs/audio-programs.tex` gives for `Program` carrying no owner.

[*TURN*]

TURN is an opaque handle for one transcribed human utterance – what a `TranscribedTurn` carries in the implementation (`src/punt_vox/voxd/conversation_mode/turn.py`). Its internal structure (the transcript text, a confidence signal) is outside this model; the state machine only needs to know a turn was detected, not what it says.

3 Free Types

Mode ::= *idle* | *listening* | *waiting* | *speaking*

Detector ::= *noDetector* | *turnDetector* | *bargeInDetector*

ZBOOL ::= *ztrue* | *zfalse*

A Z boolean (avoids the ProB/B keyword clash with `B00L`), following the convention already established in `docs/audio-programs.tex`.

4 Which Detector Governs Each Mode

$activeDetector : Mode \rightarrow Detector$ $activeDetector\ idle = noDetector$ $activeDetector\ listening = turnDetector$ $activeDetector\ waiting = turnDetector$ $activeDetector\ speaking = bargeInDetector$

This is the resolution to gap (a): a *total* function, so there is no mode for which “which detector” is left unanswered. **waiting** maps to **turnDetector**, the same detector as **listening** – the two non-speaking modes share one detector, and only **speaking** switches to **bargeInDetector**. This is not an arbitrary tie-break: barge-in is defined (FR-8) as interrupting the agent’s *speech*, and there is no speech to interrupt before **speaking** begins. Modelling **activeDetector** as a total function, rather than leaving the **waiting** case as prose, is what makes the resolution checkable – **fuzz** rejects the specification if the function is not total over **Mode**.

5 Pending Addenda

$CallState$ $mode : Mode$ $hasPendingAddendum : ZBOOL$ $hasPendingAddendum = ztrue \Rightarrow mode \in \{waiting, speaking\}$

The invariant is the second half of gap (a)’s resolution: a pending addendum — speech the turn detector captured during **waiting** that was not yet part of any turn sent to the agent — can only exist while the call has not yet returned to **listening**. Read contrapositively: whenever the call is in **idle** or **listening**, **hasPendingAddendum** is **zfalse** by construction. Every operation that lands on **listening** (**BargeIn**, **ReplyEnds**) must therefore discharge the flag explicitly, in the same operation – the model will not type-check otherwise, because the invariant would be violated on entry to **listening**. This is a formal way of saying “an addendum is never silently dropped”: it cannot survive a transition back to **listening** without the operation that makes that transition explicitly accounting for it. What the implementation does with the discharged content (fold it into the start of the next turn, per the same principle FR-9 already establishes for barge-in speech) is outside this model’s scope, but the model pins the *moment* that hand-off must happen: exactly at **BargeIn** or **ReplyEnds**, never later.

6 Initialisation

$Init$ $CallState'$ $mode' = idle$ $hasPendingAddendum' = zfalse$
--

7 Operations

7.1 Call lifecycle

$StartCall$ $\Delta CallState$ $mode = idle$ $mode' = listening$ $hasPendingAddendum' = zfalse$

A call begins listening immediately (UC-1, FR-1): there is no intermediate “setting up” mode between **idle** and **listening**.

<i>EndCall</i>
$\Delta CallState$
<i>mode</i> $\neq$ <i>idle</i>
<i>mode'</i> = <i>idle</i>
<i>hasPendingAddendum'</i> = <i>zfalse</i>

An explicit hangup (UC-9, FR-2) ends the call from any active mode, including mid-**speaking** – the human can end the call while the agent is still talking.

<i>TimeoutCall</i>
$\Delta CallState$
<i>mode</i> = <i>listening</i>
<i>mode'</i> = <i>idle</i>
<i>hasPendingAddendum'</i> = <i>zfalse</i>

The bounded-inactivity timeout (FR-2) fires only from **listening**: that is the one mode in which the human is expected to be about to speak and nothing is otherwise in flight. **waiting** and **speaking** have their own bounded-time failure paths (a synthesis call that never returns, Chapter 2’s “Known unknowns”) that are a distinct concern from mic-inactivity and are out of this model’s scope, per the PRD’s own framing of that gap as unresolved.

7.2 Turn detection

<i>TurnDetected</i>
$\Delta CallState$
<i>mode</i> = <i>listening</i>
<i>mode'</i> = <i>waiting</i>
<i>hasPendingAddendum'</i> = <i>zfalse</i>

The turn detector (§4: active in **listening**) fires end-of-turn (FR-5); the transcribed turn is sent to the agent session (**SessionAttach.send_turn**, out of this model’s scope) and the call enters **waiting** – no reply audio yet, but a turn is in flight.

<i>CaptureDuringWait</i>
$\Delta CallState$
<i>mode</i> = <i>waiting</i>
<i>mode'</i> = <i>waiting</i>
<i>hasPendingAddendum'</i> = <i>ztrue</i>

This is the operation gap (a) exists to name: the turn detector (§4: still active in **waiting**) fires again while the call is already waiting on the agent’s reply. FR-4 rules out dispatching a second, concurrent turn to the same session – one call is one human and one agent session, and the first turn has not yet been answered – so the newly-detected speech is recorded as pending, per §5, rather than sent immediately or discarded.

7.3 The agent’s turn

<i>ReplyBegins</i>
$\Delta CallState$
<i>mode</i> = <i>waiting</i>
<i>mode'</i> = <i>speaking</i>
<i>hasPendingAddendum'</i> = <i>hasPendingAddendum</i>

The reply’s first complete portion is ready to play (FR-11); the call enters **speaking**. A pending addendum, if any, carries through unchanged – it is not yet discharged, because the call has not yet returned to **listening** (§5).

BargeIn

$\Delta CallState$

mode = *speaking*

mode' = *listening*

hasPendingAddendum' = *zfalse*

The barge-in detector (§4: active in **speaking**) fires; the agent’s audio stops (FR-8) and the call returns to **listening**. Any pending addendum accumulated during the prior **waiting** phase, plus the interrupting speech itself, are both discharged here (§5) and become the start of the next turn (FR-9) – the model does not distinguish “lost because of the barge-in” from “carried forward,” because the invariant forbids the former: reaching **listening** with **hasPendingAddendum** still **ztrue** is not a reachable state.

ReplyEnds

$\Delta CallState$

mode = *speaking*

mode' = *listening*

hasPendingAddendum' = *zfalse*

The reply finishes without interruption; the call signals “ready to listen” audibly (FR-3, NFR-6) and returns to **listening**. Any pending addendum is discharged the same way as in **BargeIn** – the two operations are the only two paths into **listening** from an active call, and both carry the same obligation.

7.4 Observation

CurrentDetector

$\Xi CallState$

detector! : *Detector*

detector! = *activeDetector mode*

A non-mutating query: which detector is live right now. This is the queryable form of **gap** (a)’s resolution – a test or a caller can ask **CurrentDetector** in any reachable state and get a definite answer, never “undefined for this mode.”

8 The Single Serialized Dispatch Point

This is the resolution to **gap** (b), and it is stated in prose rather than as a further Z schema, for the same reason **voxd/programs/program.py**’s single-writer assumption is a docstring note on **Program** rather than a schema in **docs/audio-programs.tex**: Z models the shape of state and the preconditions/postconditions of a transition, not a scheduling discipline between concurrent callers. The state schema above is safe to reason about in isolation *only* if exactly one caller applies one operation at a time – the same caveat **Program** carries and does not enforce internally.

docs/conversation-mode-prd.tex names three components that can call into this state machine concurrently: turn detection, barge-in detection, and sentence-streamed synthesis (which drives **ReplyBegins**). All three are independently-running, continuous processes for the life of a call (Chapter 2, “Component overview”). Copying **Program**’s pure class shape without also copying its single-writer discipline reproduces a data race, not a design – this section is where that discipline is pinned down for Conversation Mode specifically.

Resolution: an actor with an internal, single-consumer command queue, not an explicit lock. **voxd** is fully async (**docs/conversation-mode-prd.tex**, “Sentence-streamed synthesis: one pipeline, not two” – every handler in **voxd/*.py** is `async def`), and Python’s asyncio event loop is already single-threaded and cooperative: a lock is the wrong tool for serializing access between coroutines running on one loop, because nothing preempts a coroutine mid-statement the way a thread can be preempted. An explicit lock would add ceremony (acquire/release, and the risk of holding it across an

`await` and reintroducing exactly the race it exists to prevent) for no safety a queue does not already give for free.

The concrete shape: a `CallActor` owns one `CallState` and one `asyncio.Queue` of commands (one variant per operation above – a tagged union, not a bare callable, so the actor’s dispatch loop can pattern match and each command remains a first-class, testable value per PY-DP-3). Turn detection, barge-in detection, and the synthesis pipeline each hold a reference to the queue, never to the `CallState` or the `Program`-shaped class directly – they enqueue a command and return; they never call an operation method on the state class themselves. Exactly one task drains the queue, one command at a time, applying the matching operation and awaiting nothing while doing so (the operation methods themselves, mirroring `Program`, are synchronous bare-attribute assignments with no `await` inside them) – so no other command can interleave mid-operation. This is the queue analogue of `Program`’s documented (but unenforced) single-writer assumption, made structurally impossible to violate rather than merely stated: there is exactly one reader of the queue, and the state class itself is never handed to any of the three concurrent producers.

This is a design requirement on Slice 1b’s implementation, not a component this design mission builds – consistent with this mission’s boundary (the fake and Protocol under `src/punt_vox/voxd/conversation_mode/` model *session attachment*, not the call actor). It is recorded here, in the Z specification, because `docs/conversation-mode-prd.tex` explicitly requires this document to be the place that gap is settled, not an implementation detail discovered afterward.

9 Playback Sink Ordering

Section ??’s §8 predecessor left three concurrency questions explicitly open for Slice 2a (`docs/conversation-mode-prd.tex` §??): what happens when a barge-in stop-signal races a segment finishing naturally; what happens when a second barge-in fires while the first’s buffer-clear is still in flight; and from what thread, under what discipline, a buffer clear is safe relative to a concurrent writer. This section resolves all three, by the same means §8 already resolved the analogous problem for `CallState`: not a lock, an actor with an internal, single-consumer command queue.

9.1 Basic Types and State

[*CHUNK*, *REASON*]

CHUNK is an opaque handle for one piece of synthesized audio – what a `SinkWrite` command carries in the implementation (`src/punt_vox/voxd/conversation_mode/sink_write.py`). Its internal structure (sample rate, encoding) is outside this model, exactly as *TURN*’s transcript text is outside §2’s. *REASON* is the same opaque, human-readable diagnostic type `docs/audio-programs.tex` declares for its own operations – `SinkClear`’s *reason*? is carried for logging only and, per the operation schema below, never affects the transition it labels.

SinkStatus ::= *sinkIdle* | *sinkWriting* | *sinkClosed*

<i>PlaybackSinkState</i> <i>status</i> : <i>SinkStatus</i>

Unlike `CallState`, a sink carries no pending-addendum-shaped invariant of its own – *sinkClosed* is simply terminal, enforced by every operation’s precondition below rather than by a cross-field invariant.

9.2 Initialisation

<i>SinkInit</i> <i>PlaybackSinkState</i> ' <i>status</i> ' = <i>sinkIdle</i>
--

9.3 Operations

<i>SinkWriteOp</i>
$\Delta\text{PlaybackSinkState}$
$\text{chunk?} : \text{CHUNK}$
$\text{status} \neq \text{sinkClosed}$
$\text{status}' = \text{sinkWriting}$

Appending a chunk is valid from either *sinkIdle* or *sinkWriting* – consecutive writes are the common case, not a special one – and always leaves the sink in *sinkWriting*. *chunk?* is not read by this schema beyond declaring it; the model tracks lifecycle state, not buffer contents.

<i>SinkClearOp</i>
$\Delta\text{PlaybackSinkState}$
$\text{reason?} : \text{REASON}$
$\text{status} \neq \text{sinkClosed}$
$\text{status}' = \text{sinkIdle}$

The barge-in primitive. Valid from either *sinkIdle* or *sinkWriting*, and always lands on *sinkIdle* – which is exactly what makes it idempotent: applying *SinkClearOp* to a state already at *sinkIdle* is a well-defined operation whose postcondition ($\text{status}' = \text{sinkIdle}$) already holds. This is the resolution to the second open question (stop-during-a-stop): a second *SinkClearOp* immediately following the first has a precondition that is still satisfied ($\text{sinkIdle} \neq \text{sinkClosed}$) and a postcondition that is unchanged by being applied twice. Nothing about applying *SinkClearOp* n times for any $n \geq 1$ differs, observably, from applying it once.

<i>SinkCloseOp</i>
$\Delta\text{PlaybackSinkState}$
$\text{status}' = \text{sinkClosed}$

Valid from any status, including *sinkClosed* itself (closing an already-closed sink is a no-op, matching the executable :class:‘PlaybackSink‘ Protocol’s stated contract) – the only operation with no precondition, because it is the only one a caller may always issue safely regardless of what it does not yet know about the sink’s state.

9.4 The First Open Question: Natural Completion vs. Barge-In

SinkWriteOp and *SinkClearOp* are the two operations that can race: a synthesis pipeline finishing a reply’s last segment issues one final *SinkWriteOp* at approximately the same real-world instant a barge-in detector, having just fired, issues a *SinkClearOp*. The Z schemas above do not, and structurally cannot, answer “which happens first” – that is a scheduling question, outside what a precondition/postcondition pair can express (the same limit §8 already names for *CallState*). What the schemas *do* guarantee, checked by *fuzz*, is that **every reachable ordering of the two operations lands in a well-defined state**: *SinkWriteOp* then *SinkClearOp* ends at *sinkIdle*; *SinkClearOp* then *SinkWriteOp* ends at *sinkWriting*. Neither ordering is a violated precondition, an undefined state, or data loss – the race is real, but its two outcomes are both valid, both represented in the model, and neither corrupts anything. This is why the resolution does not need to pick a winner: it only needs to guarantee the loser’s operation still applies cleanly to whatever the winner left behind, which the model proves for both orderings by construction.

9.5 The Second and Third Open Questions: Actor, Not Lock

The third open question – from what thread, under what discipline, a clear is safe relative to a concurrent writer – is not a question the pure state schema above can answer any more than *CallState* could answer §8’s equivalent. The resolution is the same one §8 already gives, reused rather than reinvented: an actor with an internal, single-consumer `asyncio.Queue` of commands. `PlaybackSinkActor` (`src/punt_vox/voxd/conversation_mode/playback_sink_actor.py`) owns one `PlaybackSink` and

one queue of `SinkCommand` values (`SinkWrite`, `SinkClear`, `SinkClose` – the executable form of *SinkWriteOp*, *SinkClearOp*, *SinkCloseOp*). Every producer – the sentence-streamed synthesis pipeline, the barge-in detector – holds a reference to the actor and calls `enqueue`; none ever calls `write`, `clear`, or `close` on the sink directly. Exactly one task drains the queue, applying one command’s `apply` coroutine to completion – awaits and all – before the next command is even dequeued.

This differs from `CallActor` in one respect worth naming precisely, because the difference is deliberate, not sloppy reuse: `CallActor`’s documented guarantee is that nothing is *awaited* inside a single `apply` (its operations are synchronous bare-attribute assignments), so an operation cannot be interrupted mid-statement even in principle. `PlaybackSinkActor`’s operations *are* coroutines – writing, clearing, and closing a real audio stream is I/O – so that stronger guarantee is unavailable. The guarantee this actor provides instead, and the one the second and third open questions actually need, is weaker but sufficient: **one command’s `apply`, including whatever it awaits internally, always runs to completion before the next command’s `apply` begins.** No two commands’ internal awaits ever interleave, because the second command is not dequeued – its `apply` does not even start – until the first has returned. This is what resolves the third open question: there is exactly one caller of `sink.write/sink.clear/sink.close` at any point in the program, always the actor’s own dispatch loop, so “which thread clears relative to which thread writes” has a single answer – the same one, every time – by construction, not by discipline the caller has to remember. It is also what resolves the second open question operationally: a stop-during-a-stop is two `SinkClear` commands enqueued close together; the actor applies the first to completion, then the second, and *SinkClearOp*’s idempotence (§9 above) means the second is a no-op that leaves the sink exactly where the first already put it.

A concrete sink implementation whose real I/O spans an OS thread outside this event loop (a PortAudio callback thread, Slice 2b’s decision) still must not receive concurrent calls from that thread and from the actor’s dispatch loop; the Protocol contract in `playback_sink.py` states this as a requirement on any conforming implementation, but discharging it is Slice 2b’s work, not this model’s – consistent with how §8 states the single-writer requirement on `CallActor`’s callers without itself being able to enforce cross-thread safety inside a concrete implementation.

10 Known Limitations

This model does not address two items `docs/conversation-mode-prd.tex` names as separately unresolved, and neither was in this mission’s scope to resolve:

- **A stuck or free-running detector.** The model assumes `activeDetector` always eventually fires the transition it governs. A detector that hangs (no transition ever fires) or free-runs (fires spuriously with no genuine speech) is a runtime failure mode this state machine cannot detect from the inside – consistent with `docs/conversation-mode-prd.tex`’s own framing of this as an open question, not a gap this document was asked to close.
- **A synthesis call that never returns mid-turn.** The model assumes `ReplyBegins` is always eventually reachable from `waiting`. A hung synthesis call is, like the stuck detector above, a bounded-time failure this document does not attempt to model; `docs/conversation-mode-prd.tex` suggests a timeout ending the turn (mirroring FR-18) as the likely mechanism, but that is implementation, not this model’s job.